

Login simples com PocketBase e Alpine.js

Um passo a passo para entender as duas telas, o formulário e a autenticação, construindo o projeto aos poucos.

O que vamos construir

Uma página de login e uma área do aluno. O navegador mostra as telas, Alpine.js cuida das interações e PocketBase confere as credenciais.

As cinco etapas

ETAPA	O QUE VAMOS APRENDER	BRANCH
01	HTML estático e navegação entre páginas	etapa-01-html-estatico
02	Alpine.js e um formulário de demonstração	etapa-02-alpine
03	Autenticação real com PocketBase	etapa-03-login-pocketbase
04	Verificação de login e saída	etapa-04-pagina-protegida
05	Organização dos arquivos do projeto	etapa-05-arquivos-organizados

Como usar este material

Abra a branch de cada etapa, leia os comentários no código, rode o projeto e faça as pequenas verificações. Cada branch mantém o trabalho das anteriores e acrescenta uma ideia nova.

Público: alunos iniciantes em HTML, CSS e JavaScript.

Antes de começar

A ideia principal é seguir o caminho dos dados. O aluno digita e-mail e senha; a página envia esses valores ao PocketBase; o PocketBase valida; se estiver tudo certo, a página abre a área do aluno.

```
aluno -> login.html -> Alpine.js -> PocketBase /users -> logado.html
```

O que cada tecnologia faz

TECNOLOGIA	PAPEL NESTE EXEMPLO
HTML	Define o conteúdo das páginas, os campos e os botões.
CSS	Controla cores, espaçamentos e aparência.
Alpine.js	Liga campos a variáveis e responde a eventos como o envio do formulário.
PocketBase	Armazena usuários e confere a senha no servidor.

Preparação do PocketBase

- Baixe o PocketBase para seu sistema em pocketbase.io e extraia o executável.
- Abra um terminal na pasta do executável e rode o comando abaixo.
- Abra o painel em http://127.0.0.1:8090/_/ e crie uma conta de administrador.
- Crie uma coleção do tipo Auth chamada users e adicione um usuário de teste com e-mail e senha.

```
.\pocketbase.exe serve
```

Preparação da página

Mantenha o PocketBase rodando. Em outro terminal, dentro da pasta do projeto, inicie o servidor de arquivos e abra a página de login.

```
py -m http.server 5500  
http://127.0.0.1:5500/login.html
```

Dica para evitar confusão

Use sempre 127.0.0.1 nos endereços deste exemplo. localhost e 127.0.0.1 podem ser tratados pelo navegador como origens diferentes e, por isso, podem ter estados de login separados.

Etapa 1 | HTML estático

Branch: etapa-01-html-estatico

Nesta primeira versão, existem duas páginas: login.html e logado.html. O foco é aprender como o HTML organiza os elementos e como um link abre outra página.

O que observar no código

- <h1> cria o título principal da página.
- <label> descreve um campo do formulário.
- <input> permite digitar um valor.
- <a href> cria um link para outra página.

```
<label for="email">E-mail</label>
<input id="email" type="email">
<a href="logado.html">Continuar</a>
```

Importante

O link Continuar não valida o e-mail nem a senha. Ele serve somente para praticar navegação. Nesta etapa, qualquer pessoa pode abrir logado.html.

Experimente

- Abra login.html e clique em Continuar sem digitar nada.
- Abra logado.html diretamente na barra de endereços.
- Mude o texto do título e veja o resultado no navegador.

Pergunta para a turma

O que precisa acontecer para o navegador saber que um aluno realmente pode entrar? A próxima etapa acrescenta interatividade, mas ainda não vai conferir uma senha real.

Etapa 2 | Alpine.js e o formulário

Branch: etapa-02-alpine

Alpine.js permite ligar HTML a pequenos trechos de JavaScript sem montar uma aplicação grande. Nesta etapa, o formulário responde ao envio e a página guarda valores temporários.

Quatro atributos para reconhecer

- x-data cria os dados e as funções disponíveis em uma parte do HTML.
- x-model mantém um campo ligado a uma variável.
- @submit.prevent chama uma função quando o formulário é enviado e impede o recarregamento padrão.
- x-show mostra ou esconde um elemento de acordo com uma condição.

```
<main x-data="loginDemo()">
  <form @submit.prevent="entrar">
    <input x-model="email">
    <p x-show="erro" x-text="erro"></p>
  </form>
</main>
```

Nesta branch, a função entrar() só verifica se os campos foram preenchidos e navega para logado.html. Não existe servidor, usuário cadastrado ou validação de senha.

Por que fazemos uma simulação?

Primeiro aprendemos a resposta do formulário. Depois substituímos a simulação pela chamada real ao PocketBase. Assim, cada etapa acrescenta uma ideia nova.

Experimente

- Envie o formulário vazio para observar a mensagem de erro.
- Preencha qualquer e-mail e senha: a simulação vai aceitar os dois.
- Procure no código x-model e acompanhe qual variável recebe cada campo.

Conceito-chave

Os dados de x-data vivem no navegador e controlam a interface. Isso não significa que o navegador esteja verificando a identidade do aluno.

Etapa 3 | Autenticação PocketBase

Branch: etapa-03-login-pocketbase

Agora o login chama o PocketBase. A coleção users precisa ser do tipo Auth, pois esse tipo de coleção inclui os recursos de autenticação.

O endereço do servidor

```
window.pb = new PocketBase('http://127.0.0.1:8090');
```

O SDK é uma biblioteca cliente. Ela sabe montar as requisições para a API do PocketBase e guardar o estado de autenticação no navegador.

A chamada de login

```
await window.pb
  .collection('users')
  .authWithPassword(this.email, this.senha);
```

- collection('users') escolhe a coleção onde estão os alunos.
- authWithPassword envia o e-mail e a senha para validação.
- await espera a resposta antes de continuar.
- try/catch separa o caminho de sucesso do caminho de erro.

Se o PocketBase aceitar as credenciais, o SDK guarda um token e a página abre logado.html. Se houver erro, a função mostra uma mensagem no formulário.

Token em poucas palavras

O PocketBase devolve um token depois do login. O SDK o conserva no armazenamento local do navegador para que as próximas páginas possam enviar pedidos autenticados.

Experimente

- Teste uma senha errada e observe a mensagem.
- Entre com o usuário criado no painel do PocketBase.
- Na etapa 3, logado.html ainda não impede que alguém a abra diretamente.

Etapa 4 | Permanecer logado e sair

Branch: etapa-04-pagina-protegida

As páginas consultam o `authStore`, o armazenamento de autenticação do SDK. Isso permite decidir qual tela mostrar e recuperar o registro do aluno.

```
if (window.pb.authStore.isValid) {  
  this.usuario = window.pb.authStore.record;  
}
```

`isValid` verifica se o token salvo ainda está dentro do prazo. `record` contém os dados do registro autenticado, como o e-mail.

A página do aluno usa a mesma ideia no sentido contrário: sem token válido, redireciona para `login.html`. Para sair, chama:

```
window.pb.authStore.clear();  
window.location.href = "login.html";
```

`clear()` apaga o estado de autenticação salvo pelo SDK neste navegador. A navegação de volta ao login encerra o fluxo da interface.

Uma distinção importante

Esconder uma página com JavaScript organiza a experiência, mas não protege dados no servidor. Dados privados precisam de regras de acesso configuradas nas coleções do PocketBase.

Roteiro de verificação

- Entre e atualize a área do aluno. O e-mail continua visível?
- Clique em Sair e abra `logado.html` diretamente. Você volta ao login?
- Confira no PocketBase as regras de acesso antes de guardar dados privados.

Etapa 5 | Organizar os arquivos

Branch: etapa-05-arquivos-organizados e branch final: main

Quando o fluxo estiver entendido, separar arquivos deixa cada assunto mais fácil de encontrar. O navegador continua abrindo duas páginas HTML.

```
login.html
logado.html
css/estilo.css
js/pocketbase.js
js/login.js
js/logado.js
```

Responsabilidade de cada arquivo

- login.html contém campos e instruções da tela de entrada.
- logado.html contém o conteúdo mostrado após o login.
- css/estilo.css guarda as regras visuais compartilhadas.
- js/pocketbase.js cria o cliente e define o endereço do servidor.
- js/login.js registra o componente Alpine e faz o login.
- js/logado.js verifica o estado, mostra o aluno e permite sair.

Ordem dos scripts

```
<script defer src="../../../pocketbase.umd.js"></script>
<script defer src="js/pocketbase.js"></script>
<script defer src="js/login.js"></script>
<script defer src="../../../alpine.min.js"></script>
```

A ordem é importante: o código de cada arquivo precisa encontrar as bibliotecas que usa. Os scripts defer são executados depois que o HTML foi lido, mantendo a ordem declarada. O componente é registrado antes de Alpine iniciar.

Como mudar de branch

No terminal, use `git switch etapa-01-html-estatico` para voltar ao início. Avance até `etapa-05-arquivos-organizados`. Use `git switch main` para retornar à versão final.

Testar e resolver problemas comuns

Antes de pedir ajuda, siga este checklist na ordem. Ele ajuda a descobrir em qual parte do caminho está o problema.

- O PocketBase está rodando no terminal? O painel abre em 127.0.0.1:8090/_/?
- A coleção se chama exatamente users e é do tipo Auth?
- O e-mail e a senha foram cadastrados nessa coleção?
- A página foi aberta por `http://127.0.0.1:5500/login.html` e não por duplo clique no arquivo?
- O endereço em `js/pocketbase.js` corresponde ao endereço onde o PocketBase está rodando?
- O computador tem internet para carregar Alpine.js e o SDK pelo CDN?

Teste manual do fluxo

- Senha errada: a tela mostra uma mensagem e permanece no login.
- Credenciais certas: a página mostra a área do aluno e o e-mail.
- Atualização da página: o login salvo continua disponível.
- Sair: o SDK limpa o token e retorna ao login.
- Acesso direto sem login: a interface volta para `login.html`.

Erros frequentes

SINTOMA	CAUSA PROVÁVEL	O QUE CONFERIR
Não foi possível conectar	PocketBase parado ou endereço diferente	Terminal e <code>js/pocketbase.js</code>
E-mail/senha inválidos	Usuário ausente ou dados incorretos	Registro na coleção users
Estilos não aparecem	Caminho do CSS incorreto	<code>css/estilo.css</code> no HTML
A tela fica invisível	Alpine não carregou ou script falhou	Internet e console do navegador

Se algo não funcionar

Leia a primeira mensagem vermelha no console do navegador e compare o caminho do arquivo com a estrutura do projeto. Mude uma coisa de cada vez.

Glossário e desafios

Palavras que aparecem no projeto

- API: uma forma definida para um programa pedir serviços ou dados a outro.
- Autenticação: conferir quem está tentando entrar.
- Auth collection: coleção especial do PocketBase que guarda contas de usuário e recursos de login.
- Estado: valores que a página está usando naquele momento, como e-mail e mensagem de erro.
- Token: credencial digital emitida depois do login e usada em pedidos autenticados.
- Redirecionamento: abrir outra URL usando JavaScript ou um link.

Desafios para continuar aprendendo

- Adicione o primeiro nome do aluno à área logada.
- Troque o texto do botão enquanto o login está acontecendo.
- Adicione um botão para voltar ao login quando já estiver autenticado.
- Leia sobre regras de API do PocketBase e restrinja uma coleção de exemplo ao usuário autenticado.
- Altere a cor principal em css/estilo.css e observe que as duas páginas mudam.

Links oficiais para consulta

[Autenticação no PocketBase](#)

[Coleções e regras do PocketBase](#)

[Instalação e início com Alpine.js](#)

Parabéns!

Você percorreu o caminho desde duas páginas estáticas até um login conectado ao PocketBase. Continue praticando uma ideia por vez e use os comentários dos arquivos como guia.